

# LAPORAN PRATIKUM

## STRUKTUR DATA

ADT Stack

disusun Oleh:

Muhammad Yasin Habiburrahman

2511532016

Dosen Pengampu:

Dr. Wahyudi. S.T.M.T

Asisten Pratikum:

Sherly Sukmadira Putri



DEPARTEMEN INFORMATIKA FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

TAHUN 2026

## **KATA PENGANTAR**

Puji dan syukur kami panjatkan ke hadirat Allah SWT atas segala rahmat dan karunia-Nya, sehingga kami dapat menyelesaikan tugas kelompok ini dengan baik. Shalawat serta salam semoga senantiasa tercurah kepada junjungan kita Nabi Muhammad SAW, keluarga, sahabat, serta para pengikutnya hingga akhir zaman.

Tugas kelompok ini kami susun dengan tujuan untuk memenuhi salah satu tugas mata kuliah Algoritma dan Pemrograman, dengan tema “ADT Stack”, yang mencakup bagaimana cara kerja dan implementasi yang digunakan untuk membuat stack.

Kami menyadari bahwa penyusunan tugas ini masih jauh dari sempurna, baik dari segi materi maupun penyajiannya. Oleh karena itu, kami sangat mengharapkan kritik dan saran yang membangun dari semua pihak demi kesempurnaan tugas kami di masa mendatang.

Akhir kata, kami mengucapkan terima kasih kepada dosen pembimbing yang telah memberikan arahan, serta kepada seluruh anggota kelompok yang telah bekerja sama dengan baik sehingga tugas ini dapat terselesaikan tepat waktu. Semoga tugas ini dapat bermanfaat bagi kami khususnya, dan bagi pembaca pada umumnya.

Padang, 15 April 2026

Muhammad Yasin Habiburrahman

## DAFTAR ISI

|                                                   |           |
|---------------------------------------------------|-----------|
| <b>KATA PENGANTAR.....</b>                        | <b>i</b>  |
| <b>DAFTAR ISI.....</b>                            | <b>ii</b> |
| <b>BAB I PENDAHULUAN.....</b>                     | <b>1</b>  |
| 1.1 Latar Belakang .....                          | 1         |
| 1.2 Tujuan .....                                  | 1         |
| 1.3 Manfaat Praktikum.....                        | 1         |
| <b>BAB II PEMBAHASAN.....</b>                     | <b>2</b>  |
| 2.1 Latihan Dasar Pemahaman Stack.....            | 2         |
| 2.2 Implementasi Stack Menggunakan Array.....     | 3         |
| 2.2.1 Konstruktorkan Validator .....              | 3         |
| 2.2.2 Method-Method Utama.....                    | 3         |
| 2.2.3 Method Tambahan .....                       | 4         |
| 2.3 Kelas Driver Stack Array.....                 | 5         |
| 2.4 Postfix Menggunakan Stack .....               | 6         |
| 2.5 Nilai Maksimum Pada Stack.....                | 7         |
| 2.6 Implementasi Stack Menggunakan ArrayList..... | 8         |
| 2.6.1 Kelas Siswa.....                            | 8         |
| 2.6.2 Kelas Stack Siswa .....                     | 8         |
| 2.6.3 Fungsi Main .....                           | 9         |
| <b>BAB III KESIMPULAN.....</b>                    | <b>12</b> |
| <b>DAFTAR PUSTAKA.....</b>                        | <b>13</b> |

# **BAB I**

## **PENDAHULUAN**

### **1.1 Latar Belakang**

Dalam ilmu komputer, pengelolaan data tidak hanya membutuhkan struktur yang mampu menyimpan elemen secara dinamis, tetapi juga struktur yang mengatur urutan akses terhadap data tersebut. Salah satu struktur data yang menerapkan aturan akses secara ketat adalah Stack atau tumpukan. Stack bekerja berdasarkan prinsip LIFO (Last In, First Out), yaitu elemen yang terakhir dimasukkan akan menjadi elemen yang pertama kali dikeluarkan, mirip seperti tumpukan piring di mana piring yang paling atas adalah yang pertama diambil.

Stack merupakan salah satu implementasi dari Abstract Data Type (ADT) yang banyak digunakan dalam berbagai aplikasi nyata, seperti mekanisme undo/redo pada aplikasi teks, penelusuran rekursi, pengecekan keseimbangan tanda kurung, hingga navigasi halaman pada browser. Dengan memahami cara kerja Stack, pengembang perangkat lunak dapat menyelesaikan berbagai permasalahan yang melibatkan pemrosesan data secara terbalik atau berbasis urutan masuk.

### **1.2 Tujuan**

1. Memahami konsep ADT Stack dan prinsip kerja LIFO yang mendasarinya.
2. Mampu mengidentifikasi dan menjelaskan operasi-operasi dasar pada Stack seperti push(), pop(), peek(), dan isEmpty().
3. Mampu mengimplementasikan ADT Stack menggunakan bahasa pemrograman Java.
4. Memahami perbedaan antara Stack dan struktur data lain seperti ArrayList dalam hal aturan akses elemen.

### **1.3 Manfaat Praktikum**

1. Praktikan mampu menerapkan konsep ADT Stack dalam menyelesaikan permasalahan pemrograman yang membutuhkan mekanisme LIFO.
2. Praktikan memiliki pemahaman yang lebih luas mengenai berbagai jenis struktur data dan kapan masing-masing struktur tersebut tepat untuk digunakan.
3. Praktikan mampu mengimplementasikan Stack secara mandiri sebagai fondasi untuk mempelajari struktur data yang lebih kompleks seperti Queue dan Tree.

## BAB II

### PEMBAHASAN

#### 2.1 Latihan Dasar Pemahaman Stack

```
1 package pekan3_2511532016;
2 import java.util.Stack;
3
4 public class latihanStack_2511532016 {
5
6     public static void main(String[] args) {
7         Stack<Integer> s = new Stack<Integer>();
8         s.push(42);
9         s.push(-3);
10        s.push(17);
11        System.out.println("nilai stack= "+s);
12        System.out.println("nilai pop = " +s.pop());
13        System.out.println("nilai stack setelah pop = "+ s);
14        System.out.println("nilai peek = "+s.peek());
15        System.out.println("nilai stack setelah peek = "+s);
16    }
17
18 }
```

Kode Program 2.1 – Kelas Latihan Stack

Sebagai permulaan praktikum, diberikan sebuah kelas sebagai pemahaman dasar untuk stack. Pada kasus ini, dideklarasikan sebuah stack dengan tipe Integer dengan nama “s”. Kemudian, dimasukkan nilai 42, -3, dan 17 secara berurutan menggunakan method push(). Lalu, isi dari stack tersebut akan dicetak.

Stack menerapkan prinsip LIFO; artinya, data yang terakhir kali masuk adalah data yang pertama kali dapat dikeluarkan. Karena 17 adalah data yang dimasukkan terakhir kali, maka dia terletak pada posisi paling atas.

Setelah itu, digunakan method pop(). Method ini akan mengembalikan sekaligus menghapus nilai paling atas dari suatu stack. Lalu, digunakan method peek() yang berguna untuk mengembalikan nilai paling atas dari suatu stack. Karena 17 sudah dihilangkan menggunakan pop(), nilai yang paling atas saat ini adalah 3. Maka method peek() mengembalikan nilai 3 saat dipanggil.

Terakhir, dicetak kembali isi dari stack setelah penghapusan nilai. Berikut adalah hasil dari program tersebut setelah dijalankan.

```
nilai stack= [42, -3, 17]
nilai pop = 17
nilai stack setelah pop = [42, -3]
nilai peek = -3
nilai stack setelah peek = [42, -3]
```

Gambar 2.1 – Hasil Kode Program Latihan Stack

## 2.2 Implementasi Stack Menggunakan Array

### 2.2.1 Konstruktor dan Validator

```

1 package pekan3_2511532016;
2
3 public class stackArray_2511532016 {
4     static final int MAX = 1000;
5     int top;
6     int a[] = new int[MAX];
7
8     boolean isEmpty_2511532016() {
9         return (top < 0);
10    }
11    stackArray_2511532016(){
12        top = -1;
13    }

```

Kode Program 2.2 – Blok Program Konstruktor dan Validator

Class ini mengimplementasikan Stack secara manual menggunakan array statis berukuran MAX = 1000. Atribut top berfungsi sebagai penunjuk indeks elemen teratas stack, dan diinisialisasi dengan nilai -1 di konstruktor sebagai tanda bahwa stack masih kosong.

Mengecek apakah stack kosong dengan memeriksa apakah  $top < 0$ . Karena top dimulai dari -1 dan bertambah setiap push, kondisi ini hanya terpenuhi ketika belum ada elemen sama sekali atau semua elemen sudah di-pop.

### 2.2.2 Method-Method Utama

```

14 boolean push_2511532016(int x) {
15     if(top >= (MAX-1)) {
16         System.out.println("Stack Overflow");
17         return false;
18     }
19     else {
20         a[++top] = x;
21         System.out.println(x + " dimasukkan ke dalam stack");
22         return true;
23     }
24 }
25 int pop_2511532016() {
26     if(top < 0) {
27         System.out.println("Stack Underflow");
28         return 0;
29     }
30     else {
31         int x = a[top--];
32         return x;
33     }

```

Kode Program 2.3 – Blok Program Push dan Pop

Method `push()` memastikan sebelum memasukkan elemen, dicek dulu apakah `top >= MAX-1`. Jika iya berarti stack sudah penuh dan terjadi **Stack Overflow**. Jika masih ada ruang, `top` di-increment terlebih dahulu baru kemudian elemen `x` disimpan di `a[top]`, sehingga elemen baru selalu menempati posisi di atas elemen sebelumnya.

Method `pop()` mengecek terlebih dahulu apakah `top < 0`. Jika iya, berarti stack kosong dan terjadi **Stack Underflow**. Jika tidak, elemen teratas disimpan dulu ke variabel `x`, kemudian `top` di-decrement, lalu `x` dikembalikan. Elemen lamanya di array sebenarnya masih ada secara fisik, tapi sudah tidak bisa diakses karena `top` sudah turun.

```

36 int peek_2511532016() {
37     if(top < 0) {
38         System.out.println("Stack Underflow");
39         return 0;
40     }
41     else {
42         int x = a[top];
43         return x;
44     }
45 }

```

Kode Program 2.4 – Blok Program Peek

Sama seperti `pop` dalam hal pengecekan underflow, bedanya `peek` hanya mengambil nilai `a[top]` tanpa mengubah nilai `top` sama sekali, sehingga isi stack tidak berubah.

### 2.2.3 Method Tambahan

```

46 void print_2511532016() {
47     for(int i = top; i > -1; i--) {
48         System.out.println(" " + a[i]);
49     }
50 }
51 }

```

Kode Program 2.5 – Blok Program Print

Mencetak isi stack dari elemen teratas ke terbawah dengan iterasi mundur dari indeks `top` sampai 0. Ini mencerminkan urutan LIFO secara visual, yaitu elemen yang paling akhir masuk ditampilkan paling atas.

### 2.3 Kelas Driver Stack Array

```

1 package pekan3_2511532016;
2
3 public class stackArrayDriver_2511532016 {
4
5     public static void main(String[] args) {
6         stackArray_2511532016 s = new stackArray_2511532016();
7         s.push_2511532016(10);
8         s.push_2511532016(20);
9         s.push_2511532016(30);
10        System.out.println(s.pop_2511532016() + " dikeluarkan dari stack");
11        System.out.println("Elemen teratas adalah :" + s.peek_2511532016());
12        System.out.println("Elemen pada stack :");
13        s.print_2511532016();
14    }
15 }

```

Kode Program 2.6 – Program Driver untuk Stack Array

Objek stack dibuat dari class stackArray yang telah dibuat sebelumnya, lalu tiga elemen dimasukkan berurutan 10, 20, 30; sehingga 30 berada di paling atas. Kemudian pop() dipanggil yang mengambil dan menghapus elemen teratas, yaitu 30, lalu peek() mengintip elemen teratas yang sekarang adalah 20 tanpa menghapusnya. Terakhir print() mencetak sisa isi stack dari atas ke bawah yaitu 20 lalu 10. Keluarannya adalah sebagai berikut.

```

10 dimasukkan ke dalam stack
20 dimasukkan ke dalam stack
30 dimasukkan ke dalam stack
30 dikeluarkan dari stack
Elemen teratas adalah :20
Elemen pada stack :
20
10

```

Gambar 2.2 – Keluaran dari Program Driver Stack Array



## 2.4 Postfix Menggunakan Stack

```

1 package pekan3_2511532016;
2 import java.util.Scanner;
3 import java.util.Stack;
4
5 public class StackPostFix_2511532016 {
6     public static int postfixEvaluate_2511532016(String expression) {
7         Stack<Integer> s_2511532016 = new Stack<Integer>();
8         Scanner input_2511532016 = new Scanner(expression);
9         while(input_2511532016.hasNext()) {
10             if(input_2511532016.hasNextInt()) {
11                 if(!s_2511532016.isEmpty()) {
12                     System.out.println(s_2511532016);
13                 }
14                 s_2511532016.push(input_2511532016.nextInt());
15             }
16             else {
17                 if(!s_2511532016.isEmpty()) {
18                     System.out.println(s_2511532016);
19                 }
20                 String operator = input_2511532016.next();
21                 int operand2 = s_2511532016.pop();
22                 int operand1 = s_2511532016.pop();
23
24                 if(operator.equals("+")) {
25                     s_2511532016.push(operand1 + operand2);
26                 }
27                 else if(operator.equals("-")) {
28                     s_2511532016.push(operand1 - operand2);
29                 }
30                 else if(operator.equals("*")) {
31                     s_2511532016.push(operand1 * operand2);
32                 }
33                 else {
34                     s_2511532016.push(operand1 / operand2);
35                 }
36             }
37         }
38
39         input_2511532016.close();
40         return s_2511532016.pop();
41     }
42     public static void main(String[] args) {
43         System.out.println("hasil postfix= "+postfixEvaluate_2511532016("5 2 4 * + 7 -"));
44     }
45 }
46

```

Kode Program 2.7 – Program Postfix

Postfix adalah cara penulisan ekspresi matematika di mana operator diletakkan *setelah* operandnya. Misal  $5 + 2$  dalam postfix jadi  $5\ 2\ +$ .

Method `postfixEvaluate()` menerima ekspresi postfix dalam bentuk String. Dibuat sebuah Stack untuk menyimpan angka, dan Scanner untuk membaca angka/operator dari String tersebut satu per satu yang dipisahkan dengan spasi.

Jika karakter yang dibaca adalah angka (`hasNextInt()`), langsung di-push ke stack. Sebelum push, isi stack dicetak dulu jika tidak kosong.

Jika karakter bukan angka, berarti ia operator. Dua elemen teratas di-pop sebagai `operand2` dan `operand1`. Kemudian, operasi dijalankan sesuai dengan operator yang didapatkan, antara (+, -, \*) menggunakan `operand2` dan `operand1`.

Setelah semua karakter habis, sisa satu elemen di stack adalah hasil akhir, hasilnya akan di-pop dan dikembalikan.

Misalkan diberikan masukan string (" $5\ 2\ 4\ *\ +\ 7\ -$ "), hasil serta proses sederhananya adalah sebagai berikut.

```
[5]
[5, 2]
[5, 2, 4]
[5, 8]
[13]
[13, 7]
hasil postfix= 6
```

Gambar 2.3 – Hasil Percobaan Postfix

## 2.5 Nilai Maksimum Pada Stack

```

3  class NilaiMaksikum_2511532016 {
4      public static int max_2511532016(Stack<Integer> s) {
5          Stack<Integer> backup = new Stack<Integer>();
6          int maxValu = s.pop();
7          backup.push(maxValu);
8          while(!s.isEmpty()) {
9              int next = s.pop();
10             backup.push(next);
11             maxValu = Math.max(maxValu, next);
12         }
13         while(!backup.isEmpty()) {
14             s.push(backup.pop());
15         }
16         return maxValu;
17     }
18     public static void main(String[] args) {
19         Stack<Integer> s = new Stack<Integer>();
20         s.push(70);
21         s.push(12);
22         s.push(20);
23         System.out.println("isi stack "+s);
24         System.out.println("Stack Teratas "+s.peek());
25         System.out.println("Nilai maksimum "+max_2511532016(s));
26     }
27 }

```

Kode Program 2.8 – Program Nilai Maksimum Menggunakan Stack

Karena stack tidak bisa diakses secara acak seperti array, satu-satunya cara membaca semua elemennya adalah dengan pop() satu per satu. Tapi masalahnya, pop() merusak isi stack asli, karena itu dibutuhkan stack cadangan (backup) untuk menampung elemen yang sudah di-pop agar bisa dikembalikan lagi nanti.

Elemen pertama yang di-pop langsung dijadikan nilai asumsi awal maxValu. Lalu selama stack belum kosong, tiap elemen di-pop, ditampung ke backup, dan dibandingkan dengan maxValu menggunakan Math.max(). Setelah semua elemen habis dibaca, isi backup di-pop kembali satu per satu dan di-push ke stack s semula. Karena backup urutannya terbalik, hasil push balik ke s akan mengembalikan urutan stack ke kondisi semula. Hasil dari contoh program maksimum stack adalah sebagai berikut.

```
isi stack [70, 12, 20]
Stack Teratas 20
Nilai maksimum 70
```

Gambar 2.4 – Hasil Program Maksimum Stack

## 2.6 Implementasi Stack Menggunakan ArrayList

### 2.6.1 Kelas Siswa

```
4 class Siswa{
5     String nama;
6     int nim;
7
8     public Siswa(String nama, int nim) {
9         this.nama = nama;
10        this.nim = nim;
11    }
12    @Override
13    public String toString() {
14        return "NIM: " + nim + ", Nama: " + nama;
15    }
16 }
```

Kode Program 2.9 – Kelas Siswa

Class sederhana dengan atribut nama dan nim, constructor untuk menginisialisasi keduanya, dan toString() yang di-override agar objek Siswa bisa dicetak dengan format yang rapi.

### 2.6.2 Kelas Stack Siswa

```
20 public class SiswaStack_2511532016 {
21     private ArrayList<Siswa> stack;
22
23     public SiswaStack_2511532016() {
24         stack = new ArrayList<>();
25     }
26     public void push(Siswa mhs) {
27         stack.add(mhs);
28     }
29     public Siswa pop_2511532016(){
30         if(!isEmpty()) {
31             return stack.remove(stack.size()-1);
32         }
33         return null;
34     }
35     public Siswa peek_2511532016() {
36         if(!isEmpty()) {
37             return stack.get(stack.size()-1);
38         }
39         return null;
40     }
41     public boolean isEmpty() {
42         return stack.isEmpty();
43     }
44     public void tampilkanSiswa_2511532016() {
45         for(int i= stack.size()- 1; i>= 0 ; i--) {
46             System.out.println(stack.get(i));
47         }
48     }
49 }
```

Kode Program 2.10 – Implementasi Kelas Siswa Menggunakan ArrayList

Kelas ini memiliki konstruktor, method-method umum pada stack, dan method tambahan yang berupa menampilkan semua isi dari stack. Stack tidak diimplementasikan dengan array atau class Stack bawaan Java, melainkan menggunakan *ArrayList*. Elemen teratas stack direpresentasikan sebagai elemen terakhir di *ArrayList*.

Karena stack saat ini diimplementasikan menggunakan ArrayList, untuk memasukkan elemen ke dalam stack, digunakan method `.add()`.

Untuk mengambil dan mengembalikan elemen terakhir pada ArrayList, digunakan method `remove()`. Sebelum itu, dicek terlebih dahulu apakah stack kosong atau tidak. Jika kosong, nilai yang akan dikembalikan adalah “null”.

Untuk mengambil nilai terakhir pada stack, pastikan apakah stack kosong atau tidak. Jika tidak, nilai bisa diambil menggunakan method `.get()` tanpa harus menghapus nilai terakhir. Jika stack kosong, akan dikembalikan nilai null.

Validator pada kelas ini sederhana, yaitu hanya wrapper dari method `isEmpty()` ArrayList.

Terakhir adalah method untuk menampilkan isi elemen dengan cara iterasi mundur, dari elemen terakhir ke elemen pertama. Sehingga elemen yang dicetak pertama adalah elemen teratas stack.

### 2.6.3 Fungsi Main

```

51 public static void main(String[] args) {
52     SiswaStack_2511532016 studentStack = new SiswaStack_2511532016();
53
54     Siswa mhs1 = new Siswa("Ali", 1);
55     Siswa mhs2 = new Siswa("Boby", 2);
56     Siswa mhs3 = new Siswa("Charles", 3);
57
58     studentStack.push(mhs1);
59     studentStack.push(mhs2);
60     studentStack.push(mhs3);
61
62
63     System.out.println("Siswa di dalam stack:");
64     studentStack.tampilkanSiswa_2511532016();
65
66     System.out.println("Siswa teratas " + studentStack.peek_2511532016());
67     System.out.println("Mengeluarkan siswa teratas dari stack: " + studentStack.pop_2511532016());
68     System.out.println("Daftar siswa setelah di pop: ");
69     studentStack.tampilkanSiswa_2511532016();
70 }

```

Kode Program 2.11 – Fungsi Main

Tiga objek Siswa dibuat dan di-push berurutan, sehingga Charles ada di paling atas. Setelah dicetak semua, `peek()` mengintip Charles, lalu `pop()` mengeluarkannya, dan stack dicetak ulang menyisakan Ali dan Bobby. Keluaran dari program tersebut adalah sebagai berikut.

```

Siswa di dalam stack:
NIM: 3, Nama: Charles
NIM: 2, Nama: Boby
NIM: 1, Nama: Ali
Siswa teratas NIM: 3, Nama: Charles
Mengeluarkan siswa teratas dari stack: NIM: 3, Nama: Charles
Daftar siswa setelah di pop:
NIM: 2, Nama: Boby
NIM: 1, Nama: Ali

```

Gambar 2.5 – Contoh Hasil Keluaran Program Stack Siswa

```

54 public static void banyakLagu_2016(ArrayList<Musik_2511532016> list) {
55     System.out.println("Banyak Lagu dalam Playlist: " + list.size());
56 }
57
58 public static void main(String[] args) {
59     ArrayList<Musik_2511532016> Playlist= new ArrayList<>();
60     Scanner sc = new Scanner(System.in);
61     int choice;
62     do {
63         tampilkanMenu_2016();
64         System.out.println("Pilih Menu: ");
65         choice = sc.nextInt();
66         sc.nextLine();
67
68         switch(choice) {
69             case 1:
70                 tambahLagu_2016(Playlist, sc);
71                 break;
72             case 2:
73                 tampilkanPlaylistLagu_2016(Playlist);
74                 break;
75             case 3:
76                 hapusLagu_2016(Playlist, sc);
77                 break;
78             case 4:
79                 banyakLagu_2016(Playlist);
80                 break;
81             case 5:
82                 System.out.println("Keluar dari program.");
83                 break;
84             default:
85                 System.out.println("Pilihan Tidak Valid.");
86         }
87
88     } while (choice != 5);
89     sc.close();
90 }
91 }

```

Fungsi Main Sebagai Loop Utama

Fungsi main menggunakan do-while dengan switch di dalamnya. Program terus berjalan sampai pengguna memilih menu 5, dan `sc.nextLine()` dipanggil setelah `nextInt()` untuk membersihkan buffer agar input String berikutnya tidak terganggu.

## Latihan

```

1 package pekan3_2511532016;
2
3 public class Website_2511532016 {
4     private String JudulWebsite_2016;
5     private String URLWebsite_2016;
6
7     //Construct
8     public Website_2511532016(String JudulWebsite_2016, String URLWebsite_2016) {
9         this.JudulWebsite_2016 = JudulWebsite_2016;
10        this.URLWebsite_2016 = URLWebsite_2016;
11    }
12
13    //Getter
14    public String getJudulWebsite() { return JudulWebsite_2016;}
15    public String getURLWebsite() { return URLWebsite_2016;}
16
17    //Setter
18    public void setJudulWebsite(String JudulWebsite_2016) {this.JudulWebsite_2016 = JudulWebsite_2016;}
19    public void setURLWebsite(String URLWebsite) {this.URLWebsite_2016 = URLWebsite;}
20 }

```

## ADT Website

```

1 package pekan3_2511532016;
2 import java.util.Stack;
3 import java.util.Scanner;
4 public class Browser_2511532016 {
5     //1. Menampilkan Menu
6     public static void tampilkanMenu_2016() {
7         System.out.println("\nMenu:");
8         System.out.println("1. Kunjungi Halaman");
9         System.out.println("2. Kembali ke Halaman Sebelumnya");
10        System.out.println("3. Melihat Halaman Saat Ini");
11        System.out.println("4. Tampilkan Banyak Halaman di History");
12        System.out.println("5. Keluar");
13    }
14    //2. Mengunjungi Website
15    public static void kunjungiWebsite_2016(Stack<Website_2511532016> history, Scanner sc) {
16        System.out.print("Masukkan Judul Website: ");
17        String judul = sc.nextLine();
18        System.out.print("Masukkan URL: ");
19        String URL = sc.nextLine();
20        history.push(new Website_2511532016(judul, URL));
21        System.out.println("Berhasil Mengunjungi Halaman!");
22    }
23    //3. Menekan tombol back
24    public static void tekanBack_2016(Stack<Website_2511532016> history) {
25        if(history.isEmpty()) {
26            System.out.println("Tidak ada halaman sebelumnya");
27        }
28        else {
29            Website_2511532016 halaman = history.pop();
30            System.out.println("Kembali dari: " + halaman.getJudulWebsite());
31        }
32    }
33    //4. Melihat halaman yang sedang aktif
34    public static void lihatHalamanAktif_2016(Stack<Website_2511532016> history) {
35        if(history.isEmpty()) {
36            System.out.println("Tidak ada halaman yang sedang dibuka!");
37        }
38        else {
39            Website_2511532016 aktif = history.peek();
40            System.out.println("Halaman saat ini: " + aktif.getJudulWebsite());
41        }
42    }
43    //5. Mengecek status history
44    public static void cekStatusHistory_2016(Stack<Website_2511532016> history) {
45        if(history.isEmpty()) {
46            System.out.println("History kosong.");
47        }
48        else {
49            System.out.println("Banyak halaman di history: " + history.size());
50        }
51    }
52 }

```

Program Simulasi Browser History dengan Implementasi stack

### **BAB III**

### **KESIMPULAN**

Berdasarkan praktikum yang telah dilaksanakan mengenai ADT Stack, dapat ditarik beberapa kesimpulan sebagai berikut.

Stack adalah struktur data yang bekerja berdasarkan prinsip LIFO (Last In, First Out), di mana elemen yang terakhir dimasukkan akan menjadi elemen yang pertama kali dikeluarkan. Operasi dasar Stack meliputi push() untuk memasukkan elemen, pop() untuk mengeluarkan elemen teratas, peek() untuk melihat elemen teratas tanpa menghapusnya, serta isEmpty() untuk mengecek apakah stack dalam kondisi kosong.

Praktikum ini menunjukkan bahwa Stack dapat diimplementasikan dengan berbagai pendekatan menggunakan class Stack bawaan Java, menggunakan array statis secara manual, maupun menggunakan ArrayList sebagai backing structure. Ketiga pendekatan menghasilkan perilaku yang sama selama operasi push dan pop mengikuti aturan LIFO, yang membuktikan bahwa Stack adalah konsep abstrak yang tidak bergantung pada satu jenis implementasi tertentu.

Selain itu, Stack terbukti berguna dalam menyelesaikan permasalahan nyata seperti evaluasi ekspresi postfix, di mana operand ditumpuk ke stack dan diproses ketika operator ditemukan, serta pencarian nilai maksimum dalam stack yang membutuhkan stack tambahan sebagai backup agar isi stack asli tetap terjaga setelah proses selesai. Secara keseluruhan, pemahaman terhadap ADT Stack memberikan dasar yang kuat untuk mempelajari struktur data lanjutan seperti Queue dan Tree.

## DAFTAR PUSTAKA

- [1]Oracle. *Java Documentation*. <https://docs.oracle.com/javase/>